[Gonzalez] Why free monads matter: https://www.haskellforall.com/2012/06/you-could-have-invented-free-monads.html
[Parsons] What does free buy us: https://www.parsonsmatt.org/2017/09/22/what_does_free_buy_us.html
[Karpov] Free monads considered harmful: https://markkarpov.com/post/free-monad-considered-harmful.html
[Serokell] Intro to free monads: https://serokell.io/blog/introduction-to-free-monads#free-algebraic-structures
[Tweag] Free monads for interpreters: https://www.tweag.io/blog/2018-02-05-free-monads/

Free monad + effect system
Free monads can be used to represent effects.
i.e. a computation represented as an **object** with a **continuation** to the next computation.

e.g. IO effect

```
-- usual functions
getLine :: IO String
putStr :: String -> IO ()

-- object representation
data Effect a where                          a = next computation
  GetLine :: (String -> a) -> Effect a       continuation (String -> a)
  PutStrLn :: String -> a -> Effect a        continuation (() -> a)
  PutStrLn :: String -> (() -> a) -> Effect a  same explicit
  deriving Functor

-- effects
type EffectF = Free Effect                   Free effect monad

getLineF :: EffectF String                   Effects
getLineF = liftF $ GetLine id

putStrLnF :: String -> EffectF ()
putStrLnF s = liftF $ PubStrLn s ()
```

```
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE GADTs #-}

import Control.Monad.Free
import Control.Monad.Trans.Writer

-- step: define simulator
data Effect a where
  PutStrLn :: String -> a -> Effect a
  GetLine :: (String -> a) -> Effect a
  deriving Functor

-- step: wrap in Free
type EffectF = Free Effect

putStrLnF :: String -> EffectF ()
putStrLnF s = liftF $ PutStrLn s ()
```

```
getLineF :: EffectF String
getLineF = liftF $ GetLine id              -- id since the ar
```

interpretations

```
-- e.g. interpret effect as IO
interpretIO :: Effect (IO a) -> IO a
interpretIO (GetLine k) = putStr "Enter value:" >> getLine >>= k
interpretIO (PutStrLn s a) = putStrLn s >> a

-- e.g. meta interpret
prettyPrint :: Effect (Writer String a) -> Writer String a
prettyPrint (PutStrLn s a) = tell ("PutStrln " <> s <> "\n") >> a
prettyPrint (GetLine next) = tell "GetLine\n" >> next ""
```

e.g. compute

```
-- step: compute with Free
c :: EffectF ()
c = do
  putStrLnF "Hello"
  putStrLnF "World"
  x <- getLineF
  y <- getLineF
  putStrLnF ("x=" <> x <> ", y=" <> y)

main :: IO ()
main = do
  iterM interpretIO c
  putStrLn $ execWriter $ iterM prettyPrint c
```

Output

```
Hello                              -- interpretIO
World
Enter value: 1
Enter value 2
x=1,y=2

PutStrln Hello                     -- prettyPrint
PutStrln World
GetLine
GetLine
PutStrln x=, y=
```

e.g.

```
c :: EffectF ()
```

```
c = do
  putStrLnF "Hello"
  putStrLnF "World"
  x <- getLineF
  y <- getLineF
  printF ("x=" <> x <> ", y=" y)


-- expanded
Free $ PutStrLn "Hello" $                putStrLnF "Hello"
 Pure ()

Free $ PutStrLn "Hello" $                putStrLnF "World
 Free $ PutStrLn "World" $
  Pure ()

Free $ PutStrLn "Hello" $                x <- getLineF
 Free $ PutStrLn "World" $
  Free $ GetLine $ id >>>
   Pure

Free $ PutStrLn "Hello" $                y <- getLineF
 Free $ PutStrLn "World" $
  Free $ GetLine $ id >>> (\x ->
   Free (GetLine $ id >>>
    Pure
   )
  )

Free $ PutStrLn "Hello" $               putStrLnF ("x=" <> x <> ",
y=" y)
 Free $ PutStrLn "World" $
  Free $ GetLine $ id >>> (\x ->
   Free (GetLine $ id >>> (\y ->
    Free (PutStrLn ("x=" <> x <> ", y=" y) $
     Pure ()
    )
   )
  )
```

Typeclass vs **free monad**
The standard way of representing effects is monad + **typeclasses**.
Typeclasses **implicitly** (i.e. magically) apply the effect.
Free monads **explicitly** apply the effect.

e.g. Typeclass

```
class Monad m => Effect m where
  doSomething :: Int -> m Int

instance Effect IO where
```

```
  doSomething = pure . (* 2)

program :: Effect m => m Int
program = do
  x <- doSomething 1
  doSomething x

main :: IO ()
main = program >>= print                              -- Implicitly
apply Effect IO instance on program
```

e.g. Free monad

```
import Control.Monad.Free

data Effect a where
  DoSomething :: Int -> (Int -> a) -> Effect a

type EffectF = Free Effect

instance Functor Effect where
  fmap f (DoSomething x k) = DoSomething x (f . k)

doSomethingF :: Int -> EffectF Int
doSomethingF x = liftF $ DoSomething x id

program :: EffectF Int
program = do
  x <- doSomethingF 1
  doSomethingF x

interpretIO :: Effect (IO Int) -> IO Int            --
Interpretation
interpretIO (DoSomething x k) = k $ 2 * x

main :: IO ()
main = iterM interpretIO program >>= print          -- Explicitly
apply interpretation
```

Basic example: multiple **interpretations**

Pass a different **interpretation** to the fold function.

e.g. iterM

```
iterM :: (Functor f, Monad m) => (f (m a) -> m a) -> Free f a -> m a

interpret1 :: f (IO a) -> IO a
interpret2 :: f (m a) ->  a
iterM interpret1 x :: IO a
iterM interpret2 x  :: Write String a
```

e.g. State operation

```
{-# HLINT ignore "Use newtype instead of data" #-}
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE RankNTypes #-}

import Control.Monad.Free
import Control.Monad.Trans.State (StateT)
import Control.Monad.Trans.State qualified as S
import Control.Monad.Trans.Writer
import Data.Function
import GHC.IORef

data State s a where
  Get :: (s -> a) -> State s a
  Put :: s -> a -> State s a
  deriving stock (Functor)

type StateF s = Free (State s)

getF :: StateF s s
getF = liftF $ Get id

putF :: s -> StateF s ()
putF s = liftF $ Put s ()

modifyF :: (s -> s) -> StateF s ()
modifyF f = getF >>= putF . f
```

interpretation 1: IO

```
interpretIO :: IORef s -> State s (IO a) -> IO a
interpretIO ref (Get k) = readIORef ref >>= k
interpretIO ref (Put s next) = writeIORef ref s >> next
```

interpretation 2: StateT

```
interpretStateT :: (Monad m) => State s (StateT s m a) -> StateT s
m a
interpretStateT (Get k) = S.get >>= k
interpretStateT (Put s next) = S.put s >> next
```

interpretation 3: pretty print

```
prettyPrint :: State Int (Writer String a) -> Writer String a
prettyPrint (Get k) = tell ("GET" <> "\n") >> k 0
prettyPrint (Put _ next) = tell ("PUT" <> "\n") >> next
```

e.g.

```
program1 :: StateF Int ()
program1 = do
  n <- getF
  putF (n + 1)
  modifyF (* 2)
  return ()

main :: IO ()
main = do
  -- interpretation 1: interpretIO
  ref <- newIORef (0 :: Int)
  iterM (interpretIO ref) program1
  s <- readIORef ref
  putStrLn $ "interpretIO state:" <> show s

  -- interpretation 2: interpretStateT
  (_, s') <- iterM interpretStateT program1 & flip S.runStateT s
  putStrLn $ "interpretStateT state:" <> show s'

  -- interpretation 3: prettyPrint
  let (_, output) = iterM prettyPrint program1 & runWriter
  putStrLn output

  return ()
```

output

```
interpretIO state:2                    -- interpretIO

interpretStateT state:6                -- interpretStateT

GET                                    -- prettyPrint
PUT
GET
PUT
```

Example: representation of computation as **objects**

Free monad **defers** the actual computation until fold.
Thus, a monadic function can be converted to free format by deferring the result to a **continuation** i.e. by using **CPS** style

```
doA :: m String                    simple monad
doB :: m ()                        no output
doC :: Int -> m String             input value
doD :: m Int -> m String           monadic input
doMap :: m x -> (x -> y) -> m y    mapping
doIO :: IO a -> m a                another monad e.g. IO

-- monad equivalent
data Do a where                          a = next value
  DoA :: (String-> a) -> Do a            continuation with Int = Int -> a
  DoB :: (() -> a) -> Do a               continuation with () = () -> a
```

DoB' :: a -> Do a	shorthand for CPS ()
DoC :: Int -> (String -> a) -> Do a	continuation with String = (String -> a)
DoD :: Int -> (String -> a) -> Do a	??? is this right?
DoMap :: x -> (x -> y) -> (y -> a) -> Do a	??? is this right?
DoIO :: IO a -> Do a	

e.g.

```
{-# LANGUAGE GADTs #-}

import Control.Monad.Free
import Control.Monad.IO.Class
import Data.Function
import Data.Functor.Classes

-- setup: original functions
doA :: MonadIO m => m String
doA = pure "A: Hello"

doB :: MonadIO m => m ()
doB = liftIO $ putStrLn "B: Hello"

doC :: MonadIO m => Int -> m Bool
doC = pure . even

doD :: (Show s, MonadIO m) => m s -> m String
doD = (show <$>)

doE :: Monad m => m a -> (a -> b) -> m b
doE = flip fmap

doIO :: MonadIO m => IO a -> m a
doIO = liftIO


-- setup: free data type for simulation
data Do a where
  DoA :: (String -> a) -> Do a
  DoB :: (() -> a) -> Do a
  DoB' :: a -> Do a
  DoC :: Int -> (Bool -> a) -> Do a
  DoD :: Show s => s -> (String -> a) -> Do a
  DoE :: x -> (x -> y) -> (y -> a) -> Do a
  DoIO :: IO a -> Do a

instance Functor Do where
  fmap f (DoA k) = DoA $ f . k
  fmap f (DoB k) = DoB $ f . k
  fmap f (DoB' a) = DoB' $ f a
  fmap f (DoC i k) = DoC i $ f . k
  fmap f (DoD s k) = DoD s $ f . k
  fmap f (DoE x g k) = DoE x g $ f . k
  fmap f (DoIO io) = DoIO $ f <$> io
```

```
instance Show1 Do where
  liftShowsPrec showA _ i (DoA f) = ("DoA " <>) . showA i (f "")
  liftShowsPrec showA _ i (DoB f) = ("DoB " <>) . showA i (f ())
  liftShowsPrec showA _ i (DoB' f) = ("DoB' " <>) . showA i f
  liftShowsPrec showA _ i (DoC _ f) = ("DoC " <>) . showA i (f
True)
  liftShowsPrec showA _ i (DoD _ k) = ("DoD" <>) . showA i (k "")
  liftShowsPrec showA _ i (DoE x f k) = ("DoE " <>) . showA i (k $
f x)
  liftShowsPrec _ _ _ (DoIO _) = ("DoIO  " <>)


--- setup: Free
type DoF = Free Do


-- setup: simulating functions
doAF :: DoF String
doAF = liftF $ DoA id

doBF :: DoF ()
doBF = liftF $ DoB id

doB'F :: DoF ()
doB'F = liftF $ DoB' ()

doCF :: Int -> DoF Bool
doCF x = liftF $ DoC x id

doDF :: Show s => DoF s -> DoF String
doDF x = x >>= liftF . flip DoD id

doEF :: DoF x -> (x -> y) -> DoF y
doEF dx f = dx >>= \x -> liftF (DoE x f id)

doIOF :: IO a -> DoF a
doIOF = liftF . DoIO


-- fold
foldDoF :: forall m a. (MonadIO m) => DoF a -> m a
foldDoF = foldFree f
  where
    f :: Do x -> m x
    f (DoA k) = k <$> doA
    f (DoB k) = k <$> doB
    f (DoB' a) = doB >> return a
    f (DoC i k) = k <$> doC i
    f (DoD s k) = k <$> doD (pure s)
    f (DoE x g k) = k <$> doE (pure x) g
    f (DoIO io) = doIO io
```

Sample computation

```
(do
    x <- doAF
    doIO $ putStrLn x
    doBF
    doB'F
    doCF 1 >>= doIOF . putStrLn . ("1 even:" <>) . show
    doCF 2 >>= doIOF . putStrLn . ("2 even:" <>) . show
    doEF (pure (2 :: Int)) (* (2 :: Int)) >>= doIOF . putStrLn .
("map:" <>) . show
    doDF (pure (1 :: Int)) >>= doIOF . putStrLn . ("pure 1:" <>)
  )
  & foldDoF


-- result
A: Hello
B: Hello
B: Hello
1 even:False
2 even:True
map: 4
pure 1:1
```

Example: use **multiple effects** (analogue of **monad transformer**)
Ref: https://markkarpov.com/post/free-monad-considered-harmful.html

Use a **wrapper effect** to combine multiple effects.
Similar to how **monad transformer** wraps monads.

data StateEffect s a = ...	effect1
data WriteEffect w a = ...	effect2
data **Effect s** w a where	**wrapper** effect
EfState :: StateEffect s a -> Effect s w a	
EfWrite :: StateEffect w a -> Effect s w a	
-- monad analogue	
StateT s (Writer w) a	analogous monad transformer
(MonadState m, MonadWriter m) =>	analogous type class

e.g. state and write effects

```
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE GADTs #-}

import Control.Monad.Free

data StateEffect s a where
  Get :: (s -> a) -> StateEffect s a
  Put :: s -> a -> StateEffect s a
  deriving stock (Functor)
```

```
data WriteEffect w a where
  Tell :: w -> a -> WriteEffect w a
  deriving stock (Functor)
```

Combine state and write effects

```
data Effect s w a where
  EfState :: StateEffect s a -> Effect s w a
  EfWrite :: WriteEffect w a -> Effect s w a
  deriving stock (Functor)

type EffectF s w = Free (Effect s w)

getF :: EffectF s w s
getF = liftF $ EfState $ Get id

putF :: s -> EffectF s w ()
putF s = liftF $ EfState $ Put s ()

tellF :: w -> EffectF s w ()
tellF w = liftF $ EfWrite $ Tell w ()
```

e.g. program

```
program :: EffectF Int String ()
program = do
  x <- getF
  putF x
  tellF $ show x
  return ()
```

Example: replace IO actions with test values (e.g. use a test value instead of getLine)
Ref: https://deque.blog/2017/11/13/free-monads-from-basics-up-to-implementing-composable-and-effectful-stream-processing/

e.g.
Effects

```
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE GADTs #-}

import Control.Monad.Free

data Effect a where
  PutStrLn :: String -> a -> Effect a
  GetLine :: (String -> a) -> Effect a
  deriving (Functor)

type EffectF = Free Effect
```

```
putStrLnF :: String -> EffectF ()
putStrLnF s = liftF $ PutStrLn s ()

getLineF :: EffectF String
getLineF = liftF $ GetLine id

-- interpretation
interpretIO :: Effect (IO a) -> IO a
interpretIO (GetLine k) = putStr "Enter value:" >> getLine >>= k
interpretIO (PutStrLn s a) = putStrLn s >> a
```

main: replace GetLine with

```
data TestAction where
  Type :: String -> TestAction

applyTestAction :: [TestAction] -> EffectF a -> EffectF a
applyTestAction [] e = e
applyTestAction _ (Pure a) = Pure a
applyTestAction ((Type s) : ts) (Free (GetLine k)) =
applyTestAction ts (k s)          -- GetLine: use test value
applyTestAction ts (Free e) = Free $ applyTestAction ts <$> e
                     -- otherwise: propagate
```

Analogous with CPS free monad

```
import Control.Monad.Free.Church

type EffectF = F Effect

applyTestAction :: [TestAction] -> EffectF a -> EffectF a
applyTestAction ts (F fld) = fld (const . pure) go ts
 where
  go :: Effect ([TestAction] -> EffectF a) -> [TestAction] ->
EffectF a
  go (GetLine k) (Type s : ts') = k s ts'
                -- GetLine: use test value
  go v ts' = wrap $ ($ ts') <$> v
                -- otherwise: propagate
```

e.g.

```
c :: EffectF ()
c = do
  putStrLnF "Hello"
  putStrLnF "World"
  x <- getLineF
  y <- getLineF
  putStrLnF ("x=" <> x <> ", y=" <> y)
```

```
-- e.g. interpret Free action as IO


main :: IO ()
main = do
  let stdInValues = (Type . show <$> [1 :: Int, 10 ..])
  iterM interpretIO $ applyTestAction stdInValues c
```

==========

OLD

```
This
is
code
Monaco 14
```

Example: Simulate state, reader, writer

Use

```
data Action w s a where                    a = dummy variable to bind Free
  -- simulate State
  Get :: (s -> a) -> Action w s a
  Put :: s -> a -> Action w s a

  -- simulate Writer
  Tell :: w -> a -> Action w s a
```

```
{-# LANGUAGE FlexibleContexts #-}
{-# LANGUAGE GADTs #-}

import Control.Monad.Free
import Control.Monad.IO.Class
import Control.Monad.RWS.Class
import Control.Monad.Trans.RWS (RWST, runRWST)

-- Step: define data
data Action w s a where
  Get :: (s -> a) -> Action w s a
  Put :: s -> a -> Action w s a
  Tell :: w -> a -> Action w s a
  deriving (Functor)


-- Step: Free constructor
type ActionF w s = Free (Action w s)

tellF :: w -> ActionF w s ()
```

```
tellF w = liftF $ Tell w ()

getF :: ActionF w s s
getF = liftF $ Get id

putF :: s -> ActionF w s ()
putF s = liftF $ Put s ()



-- Step:
foldRWS :: (MonadState s m, MonadWriter w m, MonadIO m) => ActionF
w s a -> m a
foldRWS = iterM f
  where
    f (Get k) = fPutStr "Get:" >> get >>= k
    f (Put s a) = put s >> fPutStr "Put:" >> a
    f (Tell w a) = tell w >> a
    fPutStr msg = liftIO $ putStrLn msg
```

e.g.

```
c :: ActionF String Int ()
c = do
  tellF "INFO: Start"
  getF >>= (tellF . ("current state:" <>)) . show
  getF >>= putF . (+ 100)
  getF >>= (tellF . ("updated state:" <>)) . show
  tellF "INFO: end"
  pure ()

(((), lg), state) <- runState (runWriter $ foldRWS c) 0    --
initial state = 0, final state = 100
                                                           -- lg
= "INFO: Start current state: 0 updated state: 100 INFO: end"
```